

Phase -1.5 — High-Level System Architecture

Objective

The Functional Requirements define **what** the system should do.

The User Stories define **who** performs those actions.

This phase answers the next question:

How should the entire system be organized to support those requirements?

The High-Level Architecture (HLD) focuses on the major building blocks of the application and how they communicate with one another. It intentionally avoids implementation details such as classes, methods, or database schemas.

What is High-Level Design (HLD)?

High-Level Design describes the overall structure of a software system.

It identifies:

- Major components
- Communication between components
- External systems
- Data flow
- Deployment boundaries
- Technology boundaries

Think of it as the blueprint of a building before deciding where every electrical wire or plumbing pipe goes.

Architectural Goals

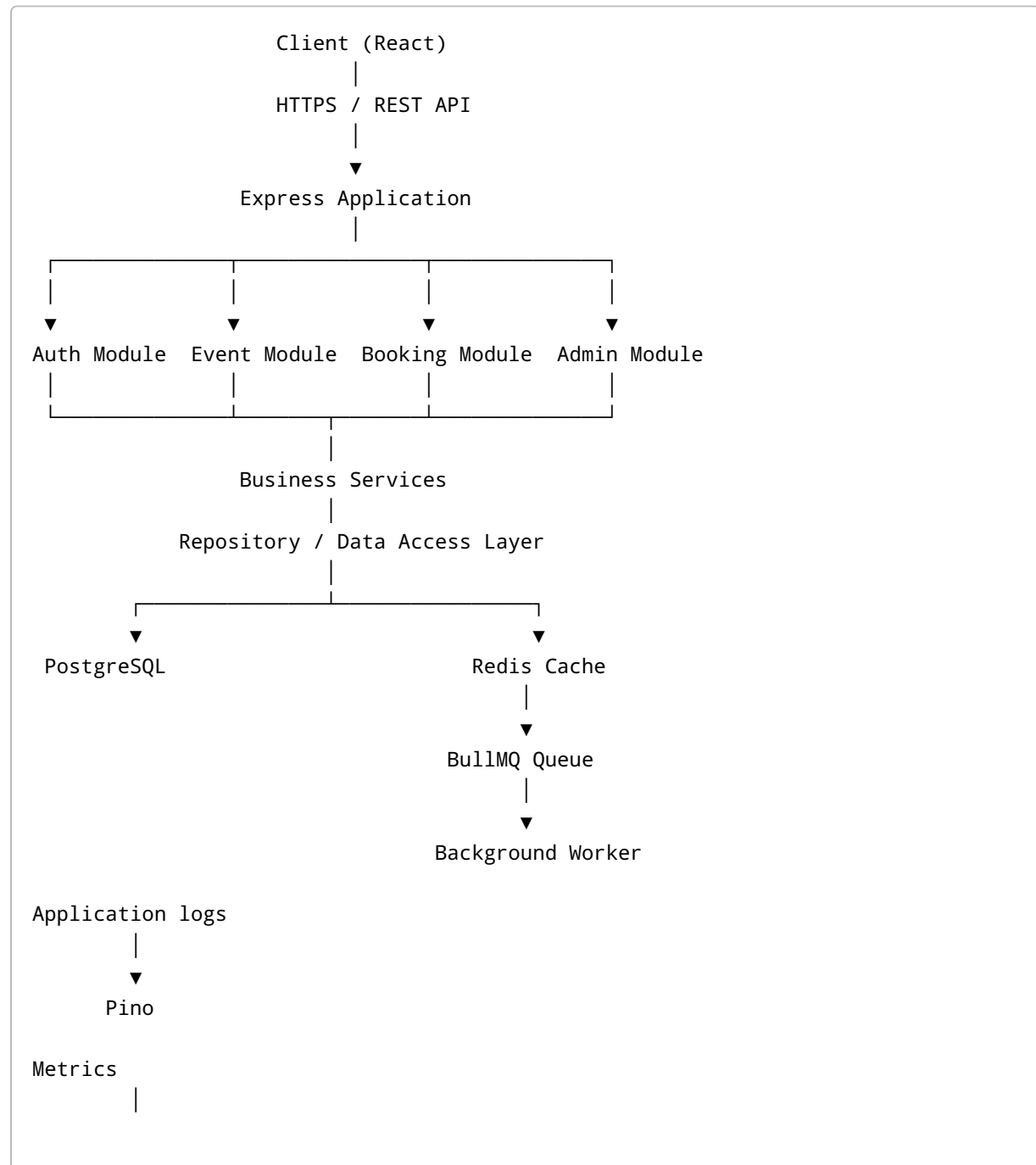
The architecture should satisfy the following goals.

- Modular
- Maintainable
- Testable
- Scalable
- Secure
- Observable

- Easy to extend
- Production-oriented

Every architectural decision made later should support one or more of these goals.

Overall System Architecture



Core Components

Client Application

Responsibility

Provides the graphical interface used by customers and administrators.

Responsibilities include:

- Authentication
- Displaying events
- Selecting seats
- Booking tickets
- Viewing bookings

The frontend contains **no business logic**.

It only communicates with backend APIs.

API Layer

Responsibility

The Express server exposes REST APIs to external clients.

Responsibilities include:

- Routing
- Request validation
- Authentication
- Authorization
- Response generation
- Error handling

No business logic should exist inside controllers.

Business Layer

Responsibility

The Service Layer contains all business rules.

Examples include:

- Booking validation
- Seat reservation
- Payment processing
- Authentication
- Authorization
- Event management

This is the heart of the application.

Almost every business decision belongs here.

Repository Layer

Responsibility

The Repository Layer communicates with PostgreSQL.

Responsibilities include:

- Reading data
- Writing data
- Transactions
- Query abstraction

Business logic should never exist inside repositories.

Repositories answer **how data is stored**, not **how the business works**.

PostgreSQL

Responsibility

Primary source of truth.

Stores:

- Users
- Events
- Seats
- Bookings
- Payments
- Refresh Tokens

Every permanent piece of business data belongs here.

Redis

Responsibility

Provides fast in-memory storage.

Used for:

- Frequently accessed event data
- Distributed locks (future)
- BullMQ
- Temporary application data

Redis never replaces PostgreSQL.

It complements it.

BullMQ Worker

Responsibility

Processes asynchronous work.

Examples include:

- Seat expiration
- Booking confirmation
- Retry jobs

This keeps slow operations outside the HTTP request lifecycle.

Logging System

Responsibility

Records application activity.

Every important action should be logged.

Examples:

- User login
- Booking creation
- Payment result
- Background jobs
- Unexpected exceptions

Logging should never replace debugging.

It provides visibility into running systems.

Monitoring System

Responsibility

Provides operational insight.

Metrics include:

- Request count
- Error rate
- Response time
- Booking throughput

These metrics allow engineers to understand system health.

Environment Strategy

Objective

The application should behave differently depending on where it is running, while keeping the source code identical.

The only thing that changes between environments should be **configuration**, not code.

Supported Environments

Development

Purpose:

Local development.

Characteristics:

- Hot reload
- Verbose logging
- Swagger enabled
- Source maps enabled
- Local PostgreSQL
- Local Redis
- Mock payment provider

Production

Purpose:

Real users.

Characteristics:

- Optimized builds
- Minimal logging
- Secrets from Secret Manager
- HTTPS only
- Monitoring enabled
- Metrics enabled
- Rate limiting enabled
- Debug endpoints disabled

Environment Configuration

Instead of:

```
if (development) {  
    ...  
}
```

everything should come from configuration.

Example:

```
NODE_ENV=development  
  
DATABASE_URL=...  
  
REDIS_URL=...  
  
JWT_SECRET=...  
  
LOG_LEVEL=debug
```

The application should never need to know where it is running beyond its configuration.

Configuration Loading

The configuration layer should become the single source of truth.

Nothing else in the application should read environment variables directly.

Instead:

```
Environment Variables  
  |  
  ▼  
Configuration Module  
  |  
  ▼  
Entire Application
```

Secrets Management

Development

.env

Testing

.env.test

Production

Never committed.

Instead use

- AWS Secrets Manager
- Azure Key Vault
- Google Secret Manager
- Kubernetes Secrets

Even though we'll use `.env` locally, the architecture should assume secrets come from a secure secret manager in production.

Build Strategy

Development

TypeScript

↓

ts-node-dev

Production

TypeScript

↓

Compiled JavaScript

↓

Node.js

Never run TypeScript directly in production.

Logging Differences

Development

Pretty logs

Production

Structured JSON logs

Error Handling Differences

Development

Return detailed stack traces.

Production

Return generic error messages while logging the complete stack internally.

API Documentation

Development

Enabled.

Production

Usually disabled or protected.

Cross-Cutting Components

Some components affect every module in the application.

These are known as **cross-cutting concerns**.

Authentication

Responsible for identifying the user.

Every protected endpoint depends on it.

Authorization

Determines whether an authenticated user has permission to perform an operation.

Validation

Every incoming request must be validated before reaching business logic.

Invalid requests should fail immediately.

Global Error Handling

All application errors should be handled by a centralized error handler.

Responsibilities include:

- Mapping exceptions to HTTP responses
- Logging unexpected failures
- Returning consistent error responses
- Preventing stack traces from reaching clients

Controllers should never manually format error responses.

Logging

Every request should generate structured logs.

Logs should include:

- Request ID
 - Timestamp
 - User ID (when available)
 - Route
 - Processing time
 - Status code
-

Configuration

Application configuration should be centralized.

Examples include:

- Database URL
- Redis URL
- JWT secrets
- Environment variables
- Feature flags

Configuration should never be hardcoded.

Request Lifecycle

Every request follows the same path.



This predictable request flow greatly improves maintainability and debugging.

Why Layered Architecture?

Several architectural patterns exist.

Examples include:

- Layered Architecture
- Clean Architecture
- Hexagonal Architecture
- Onion Architecture
- Vertical Slice Architecture

For this project, we will use a **Layered Modular Architecture**.

Reasons:

- Easy to understand.
- Common in industry.
- Excellent for learning.
- Simple to test.
- Easy to extend.

As the project grows, we will gradually incorporate ideas from Clean Architecture without introducing unnecessary complexity.

Future Evolution

Although this project begins as a modular monolith, the architecture intentionally supports future evolution.

Possible future enhancements include:

- Separate Authentication Service
- Separate Booking Service
- Payment Microservice
- Notification Service
- API Gateway
- Message Broker
- Distributed Cache
- Kubernetes Deployment

The initial architecture should not prevent future decomposition.

Architecture Principles

Every implementation should follow these principles.

1. Single Responsibility Principle.
 2. Separation of Concerns.
 3. Dependency Inversion.
 4. Stateless APIs.
 5. Fail Fast.
 6. Explicit Error Handling.
 7. Immutable Domain Rules.
 8. Infrastructure should remain replaceable.
 9. Business logic should be framework-independent wherever practical.
 10. Simplicity is preferred over unnecessary abstraction.
-

Deliverables

By the completion of this phase, the system architecture is fully defined.

We now understand:

- The major system components.
- Their responsibilities.
- How requests flow through the application.
- Cross-cutting concerns.
- External dependencies.
- Architectural boundaries.
- Future scalability path.

The next phase, **Phase -1.6 — Low-Level Design (LLD) & Module Breakdown**, will decompose each high-level component into its internal modules, layers, responsibilities, and interactions, providing the detailed blueprint that will directly guide implementation.